Modelagem de banco de dados · Conceitos essenciais (v2)

Preparação para modelar o banco do Revista Express a partir das decisões D-046 a D-087 (Django + PostgreSQL)

Objetivo e ordem de estudo

Ao final você consegue: justificar cada chave, cada UNIQUE e cada `on_delete` do modelo em `docs/Modelo_ER_Revista.pdf`; dizer o que o banco deve guardar e o que deve calcular; explicar como o ano, as sobrescritas do projeto, a foto da revista e o bloqueio de escrita viram tabelas e restrições.

Sessão A (um dia): Parte 1, seções 1 a 6, cerca de 30 min. **Sessão B** (outro dia): Parte 2, seções 7 a 13, cerca de 35 min, depois a prática no papel (15 min) e as questões (30 min). Os tempos são estimativas de leitura.

Se a atenção cair, pare no fim de qualquer seção. Cada uma termina num ponto de parada.

Por que esta versão existe

A v1 foi escrita antes do levantamento de 27/10. Cinco coisas mudaram e cada uma tem lugar aqui:

1. A herança viva entre projetos saiu (D-051). O projeto só aponta para uma matriz e sobrescreve três coisas: blocos de texto, cores e intervalo mínimo. Isso é a **seção 8**, que substitui a antiga seção de herança e a CTE recursiva.

2. Os perfis são globais (D-073): não há mais tabela Membro por projeto. **Seção 11**.

3. O ano virou dimensão de quase tudo (D-046, D-086). **Seção 7**.

4. Grupo de corte, etapas avaliadas e padrão da tarefa são calculados, não guardados (D-049, D-058). **Seção 4**, com os erros reais da sua planilha.

5. O aviso ao impresso deixou de ser um relatório do histórico e virou foto da revista + hash (D-077). **Seção 9**.

Os exemplos com slug, cadernos e Mongo saíram: o ano de 2026 começa do zero (D-076).

Como os trechos foram verificados

Todo código deste material rodou em PostgreSQL 17 com Django 6.1.2, num app de laboratório com as tabelas do rascunho. Os resultados citados (mensagens de erro, números de grupo, hashes) são a saída real dessa execução.

PARTE 1 · Ferramentas

1. Três níveis de modelo · 4 min

Modelar é decidir quais fatos o sistema lembra, como eles se ligam e quais regras o banco impõe sozinho. O erro mais comum é começar pelas tabelas. Comece pelas perguntas: o que existe, quem cria e altera, o que muda com o tempo e o que nunca pode acontecer. A

modelagem tradicional separa a conversa em três níveis, nesta ordem.

Nível	Pergunta	Artefato	Exemplo no Revista Express
Conceitual	O que existe e quais regras valem?	Entidades e relacionamentos na língua do negócio	Parceiros com cortes iguais formam um grupo.
Lógico	Como vira tabelas, colunas e chaves?	Tabelas com PK, FK e restrições, sem SGBD definido	Grupo não é coluna: sai de ordenar os cortes distintos.
Físico	Como fica num SGBD real?	Tipos, índices, arrays, JSONB; no Django, models.py e migrations	Corte.valores como `integer[]` com CHECK de 1 a 4 elementos.

Pare e pense

«Se o parceiro não tem corte numa etapa × CC, ele não avalia aquela etapa.» Essa frase pertence a qual nível? *Responda mentalmente antes de ler o próximo parágrafo.*

É conceitual: é uma regra do negócio, dita pela equipe. A decisão de *não* criar uma tabela de participação e deduzir a participação da existência de uma linha em Corte é lógica. Uma discussão de modelagem anda em círculos quando mistura os níveis; feche o conceitual antes de discutir tipo de coluna.

2. Entidades, relacionamentos e cardinalidade · 5 min

Entidade é uma coisa com identidade própria sobre a qual o sistema guarda fatos: Parceiro, Matriz, Projeto. Atributo é um fato sobre ela (nome, escala máxima). Para decidir entre os dois, pergunte se o conceito tem vida própria: tem filhos, é referenciado por várias tabelas, muda de forma independente ou precisa de histórico.

Tipo	Leitura	Como vira tabela	Exemplo
1:N	Uma matriz tem várias MatrizEtapaCC; cada uma pertence a uma matriz	FK no lado N	MatrizEtapaCC.matriz_id
1:1 ou 0..1	Um projeto tem no máximo um bloqueio	FK com UNIQUE no lado dependente	Bloqueio.projeto_id UNIQUE
N:N	Um parceiro tem cortes em várias etapas; uma etapa tem cortes de vários parceiros	Tabela associativa	Corte(parceiro, ano, etapa, cc)

A opcionalidade é o mínimo da cardinalidade. «Toda tarefa pertence a um descritor» vira FK `NOT NULL`. «O intervalo mínimo pode não existir» (2EF) vira coluna que aceita NULL, e aqui o NULL tem significado de negócio: sem linhas vazias na escala.

Pare e pense

Corte é entidade ou só a ligação entre Parceiro e Etapa? *Responda mentalmente antes de ler o próximo parágrafo.*

É entidade. A ligação carrega atributos (os valores dos cortes), tem identidade própria (parceiro × ano × etapa × CC) e outras regras dependem dela: o projeto nasce dela e a revista só mostra as etapas que a têm. Quase todo N:N do mundo real é assim: quando a ligação tem atributos, ela é uma tabela de primeira classe, não um detalhe.

3. Chaves e escopo · 5 min

A chave primária (PK) identifica a linha: única, não nula e imutável. A chave estrangeira (FK) aponta para a PK de outra tabela, e o banco garante que o destino existe. Chave natural é um valor do negócio («2EF», «D01», «GO»); chave substituta é um número sem significado (o `id` que o Django cria). Use a substituta como PK e proteja a natural com UNIQUE: valores do negócio mudam, e uma PK que muda obriga a reescrever todas as FKs.

No seu banco quase toda chave natural tem **escopo**. «2EF» não é único no sistema: existe em 2026 e em 2027 (D-086). «D01» existe em cada MatrizEtapaCC. Por isso a restrição é composta:

```
class Etapa(models.Model):
    ano = models.ForeignKey(Ano, on_delete=models.PROTECT)
    codigo = models.CharField(max_length=4)            # "2EF"
    escala_max = models.PositiveSmallIntegerField()     # 1000 no 2EF, 500 nas demai
s
    class Meta:
        constraints = [models.UniqueConstraint(
            fields=["ano", "codigo"], name="etapa_unica_no_ano")]

# 2EF repetida em 2026 -> IntegrityError: ... unique constraint "etapa_unica_no_ano"
# 2EF em 2027            -> aceita
```

O segundo padrão é a unicidade só entre linhas num certo estado. Um parceiro tem no máximo um projeto *ativo* por ano (D-047), mas pode ter projetos arquivados (D-068). O PostgreSQL resolve com índice único parcial:

```
class Meta:
    constraints = [models.UniqueConstraint(
        fields=["parceiro", "ano"], condition=Q(arquivado=False),
        name="um_projeto_ativo_por_parceiro_e_ano")]

# 2º projeto ativo GO 2026      -> IntegrityError
# arquivar o 1º e criar outro   -> aceito; ativos = 1
```

> **Pare e pense**
>
> Por que `UNIQUE(codigo)` em Etapa quebraria a clonagem do ano? *Responda mentalmente antes de ler o próximo parágrafo.*

Porque clonar 2026 para 2027 copia a linha «2EF». Com UNIQUE só no código, a cópia viola a restrição e a transação inteira volta. A pergunta útil para qualquer tabela é «o que torna duas linhas a mesma linha?». No seu banco, a resposta quase sempre inclui o ano, direta ou indiretamente (via matriz).

4. Normalização, dado derivado e arrays · 7 min

Normalizar é guardar cada fato uma vez, no lugar de que ele depende. A fórmula tradicional: todo atributo depende da chave, da chave inteira e de nada além da chave.

Forma	Regra	Violação no seu domínio
1FN	Valor atômico por célula; sem colunas repetidas	`corte1, corte2, corte3, corte4` como colunas
2FN	Com chave composta, nada depende só de parte dela	Corte(parceiro, ano, etapa, cc, valores, escala_max): a escala depende só da etapa
3FN	Atributo não-chave não depende de outro não-chave	Projeto(matriz_id, matriz_nome)

Há um caso que as formas normais não nomeiam bem e que é o mais importante aqui: o **dado derivado**. Se um valor pode ser calculado a partir de outros guardados, guardá-lo cria uma segunda fonte da verdade que vai divergir. A sua própria planilha mostra isso. O número do grupo foi digitado à mão e saiu errado em dois lugares: no 9EF-MT, SE (250, 300, 350) ficou no grupo 5 junto com (225, 300, 350); no 5EF-MT, SALVADOR (200, 235, 253) ficou no grupo 5 de MT (175, 225, 300).

```
distintos = sorted({tuple(v) for v in cortes.values()})   # compara corte a corte
grupo = {v: i for i, v in enumerate(distintos, start=1)}

# 9EF-MT: MS 1, MT 2, PE 3, CE 4, BA 5, SE 6, SALVADOR 7
# 5EF-MT: MS 1, PE 2, CE 3, AC 4, MT 5, SALVADOR 6
# 2EF-MT: MS 1, MT 2, SE 3, AC 4, GO/MG 5, PE 6, AM/MA 7
```

Os três resultados batem com as suas correções e com a aba 2EF-MT. Pela mesma razão não viram coluna: as etapas avaliadas (existência de Corte) e o padrão de cada tarefa (intervalo da tarefa contra os cortes do parceiro, D-058).

> **Pare e pense**
>
> Corte.valores como array de inteiros viola a 1FN? *Responda mentalmente antes de ler o próximo parágrafo.*

Pela leitura estrita, sim: a célula guarda uma lista. A alternativa normalizada é CorteValor(corte, posicao, valor). O array é defensável aqui porque a lista é curta (1 a 4), sempre lida e gravada inteira, tem ordem com significado, nenhuma FK aponta para um valor isolado e a regra de tamanho cabe num CHECK. A troca é consciente: a regra «crescente» e a regra «dentro de 0..escala_max» não cabem em CHECK e vão para a validação da aplicação. Prerequisitos do descritor (lista de textos exibida inteira) seguem o mesmo raciocínio.

Critério	Tabela filha	Array (`integer[]`, `text[]`)	JSONB
Filtrar ou indexar elemento	Índice normal	Índice GIN	Índice GIN
FK sobre o elemento	Sim	Não	Não
Regra de tamanho	Contagem com trigger	CHECK com `valores__len`	Não, sem função própria
Estrutura varia por linha	Não	Não (tipo único)	Sim
Uso no seu modelo	Tarefa, BlocoTexto	Corte.valores, prerequisitos	FotoRevista.conteudo

5. Integridade, exclusão e transações · 6 min

O banco é a última linha de defesa. O código da aplicação tem bugs, e outros clientes (um script de importação do PDF, o admin do Django, um SQL rodado à mão) não passam pela validação dela. NOT NULL, UNIQUE, CHECK e FK rodam sempre. Do ponto de vista de segurança: validação no front-end não é controle, validação na aplicação não cobre os outros clientes, restrição no banco é garantia.

O limite que você precisa conhecer: um **CHECK só enxerga colunas da própria linha**. Ele barra `inicio >= fim` numa tarefa e conta os elementos de um array, mas não consegue comparar o corte com a `escala_max` da etapa, que está em outra tabela. Para regras entre tabelas há três caminhos, do mais simples ao mais forte: validação em `clean()` (só vale para quem passa pelo Django), FK composta (quando a regra é «as duas FKs concordam», seção 7) e trigger (qualquer regra, mais código para manter).

on_delete	Efeito ao apagar o pai	No seu modelo
CASCADE	Apaga os filhos junto	Descritor → MatrizEtapaCC; Tarefa → Descritor; sobrescritas → Projeto
PROTECT	Recusa enquanto houver filhos	Matriz e TextoModelo usados por projeto; Ano; Parceiro; Etapa
SET_NULL	Filho fica com FK nula	Autor no histórico (django-simple-history)

Verificado: apagar a matriz comunitária com um projeto apontando para ela lança `ProtectedError`. O projeto não se apaga: ele é arquivado (D-065). O usuário também não se apaga: o histórico guarda o autor com SET_NULL, e apagar o usuário apagaria a autoria de tudo o que ele editou; desative em vez de apagar.

Uma transação agrupa operações que valem tudo ou nada (`transaction.atomic()`). Você vai precisar dela em três lugares: clonar um ano, salvar cortes que criam ou arquivam projetos em cascata, e adquirir o bloqueio.

> **Pare e pense**
>
> Alguém roda `DELETE FROM etapa WHERE codigo = '2EF'` direto no psql. O PROTECT do Django impede? *Responda mentalmente antes de ler o próximo parágrafo.*

Não. CASCADE, PROTECT e SET_NULL são executados pelo Django em Python, antes do DELETE; a FK que ele cria no PostgreSQL não tem ação de exclusão (no lab, `confdeltype`

= 'a', NO ACTION). Quem barra o DELETE cru é essa FK: com cortes apontando para a etapa, o banco recusa por violação de chave estrangeira (verificado). O efeito parece o do PROTECT, mas por outro motivo, e um CASCADE do Django também não acontece no psql. O Django 6.1 trouxe `DB_CASCADE`, `DB_SET_NULL` e `DB_SET_DEFAULT`, que gravam a ação no próprio banco; não há um `DB_PROTECT`, porque o NO ACTION já protege.

6. Do modelo ao Django · 3 min

Conceito	Em Django (verificado no 6.1.2)
Entidade	`class Corte(models.Model)`
1:N	`ForeignKey(Ano, on_delete=models.PROTECT)`
UNIQUE composto ou parcial	`UniqueConstraint(fields=[...], condition=Q(...), name=...)`
CHECK	`CheckConstraint(condition=Q(...), name=...)`
Exatamente um de dois	`Q(a__isnull=False) ^ Q(b__isnull=False)` dentro do CHECK
Array PostgreSQL	`ArrayField(models.PositiveSmallIntegerField())` (app `django.contrib.postgres`)
Tamanho do array	`Q(valores__len__gte=1, valores__len__lte=4)`
JSONB	`JSONField()`
Histórico por linha	`history = HistoricalRecords()` (django-simple-history)
Regra que o ORM não declara	`migrations.RunSQL(...)` na migration (ex.: FK composta)

A frase que resume o dia

Guarde cada fato uma vez, calcule o que pode ser calculado e deixe no banco toda regra que não pode ser quebrada.

PARTE 2 · Os problemas deste banco

7. O ano como dimensão · 8 min

Tudo pode mudar de um ano para outro (D-046), então quase toda tabela pertence a um ano, direta ou indiretamente: Etapa, Componente, CorPadrao, Matriz, TextoModelo, Corte e Projeto têm `ano_id`; MatrizEtapaCC, Descritor e Tarefa herdam o ano pela matriz. Parceiro é a exceção: GO é o mesmo parceiro em todos os anos.

Ter o ano em várias tabelas cria um risco novo: duas FKs da mesma linha apontarem para anos diferentes. Um Projeto de 2027 com `matriz_id` de uma matriz de 2026 é aceito por FKs simples, e a revista de 2027 sai com o conteúdo de 2026 sem erro nenhum. A FK composta faz o banco exigir que as duas concordem:

```
ALTER TABLE matriz  ADD CONSTRAINT matriz_id_ano UNIQUE (id, ano_id);
ALTER TABLE projeto ADD CONSTRAINT projeto_matriz_mesmo_ano
    FOREIGN KEY (matriz_id, ano_id) REFERENCES matriz (id, ano_id);

-- projeto 2027 com matriz 2026 -> IntegrityError: ... violates foreign key
--                                  constraint "projeto_matriz_mesmo_ano"
```

O `UNIQUE (id, ano_id)` parece redundante (o id já é único), mas é exigido: uma FK só pode apontar para colunas com UNIQUE ou PK. O mesmo par vale para Projeto × TextoModelo, Corte × Etapa e MatrizEtapaCC × Etapa. O `ForeignKey` do Django não declara isso; no lab a restrição entrou por SQL, e no projeto real entra por RunSQL numa migration.

Clonar o ano (D-054) é copiar um grafo de tabelas trocando as FKs pelas cópias novas. A ordem é a das dependências: primeiro quem não aponta para ninguém do ano, por último as folhas. Cada cópia devolve um mapa id antigo → objeto novo, usado pelas tabelas seguintes. Tudo numa transação: se a 5ª tabela falhar, as 4 primeiras voltam.

```
def copiar(qs, troca):
    mapa = {}
    for o in qs:
        antigo = o.pk
        o.pk = None; o._state.adding = True     # vira linha nova
        troca(o)                                # aponta FKs para as cópias
        mapa[antigo] = o
    qs.model.objects.bulk_create(mapa.values()) # no PostgreSQL, preenche os ids
    return mapa

@transaction.atomic
def clonar_ano(origem, destino):
    novo = Ano.objects.create(ano=destino)
    et = copiar(Etapa.objects.filter(ano=origem), lambda n: setattr(n, "ano", novo))
    ...  # Componente, Matriz, MatrizEtapaCC, Descritor, Tarefa, Corte

# resultado: 2026 e 2029 com as mesmas contagens; Projeto 2029 = 0;
# a tarefa clonada aponta para matriz e etapa de 2029
```

O lab clonou Etapa, Componente, Matriz, MatrizEtapaCC, Descritor, Tarefa e Corte. Num clone completo entram também TextoModelo, BlocoTexto e CorPadrao, que são por ano (D-086). Não entram Projeto e suas sobrescritas (D-085), FotoRevista e Bloqueio.

Pare e pense

O lab tentou apagar um ano clonado com `Ano.objects.filter(ano=2028).delete()`. O que aconteceu, e é isso que você quer? *Responda mentalmente antes de ler o próximo parágrafo.*

Lançou `ProtectedError` citando Etapa, Componente, Matriz e Corte. É o comportamento desejado: apagar um ano inteiro com um clique não pode ser acidente. Desfazer um clone errado vira uma operação explícita, que apaga na ordem inversa da cópia, dentro de uma transação, e de preferência só enquanto o ano ainda não tem projetos.

8. Padrão com sobrescrita (substitui a herança viva) · 7 min

O projeto não herda de outro projeto. Ele lê a matriz e o texto-modelo e sobrescreve três coisas: o intervalo mínimo por etapa × CC (D-053), a cor de um padrão (D-064) e um bloco de texto inteiro (D-062). As três seguem o mesmo padrão: **valor efetivo = sobrescrita, se existir; senão, o padrão**. A sobrescrita mora numa tabela própria, e a existência da linha é o que diz «sobrescrevi».

```
sobrescrita = ProjetoIntervaloMinimo.objects.filter(
    projeto=p, etapa=OuterRef("etapa"), cc=OuterRef("cc")).values("intervalo_minimo"
)
MatrizEtapaCC.objects.filter(matriz=p.matriz).annotate(
    efetivo=Coalesce(Subquery(sobrescrita), "intervalo_minimo"))

# matriz 125, projeto sobrescreveu 100 -> efetivo 100
```

Por que tabela e não uma coluna anulável no projeto? Com tabela, «não sobrescrevi» é a ausência da linha, e um valor vazio de propósito continua possível (`conteudo = ''` num ProjetoBloco). Com coluna anulável, NULL precisa significar «herda», e o vazio intencional exige outro marcador. A distinção NULL × vazio da v1 continua valendo, só mudou de lugar.

Os blocos de texto têm um passo a mais: a sobrescrita guarda a `versao` do bloco no momento em que foi feita (`versao_base`). Quando o modelo muda, a versão sobe, e a consulta abaixo lista o que ficou para trás. Pelo D-087, isso só avisa; não bloqueia o «Validado».

```
ProjetoBloco.objects.filter(projeto=p, versao_base__lt=F("bloco__versao"))
# depois de editar a Apresentação no modelo: [('Apresentação', 1, 2)]
```

Pare e pense

O intervalo mínimo da matriz muda de 125 para 150. O projeto que sobrescreveu com 100 fica sabendo? *Responda mentalmente antes de ler o próximo parágrafo.*

Não. Ele continua com 100 e nada no modelo atual registra que o padrão mudou por baixo dele. Os blocos têm `versao_base`; intervalo mínimo e cor não têm. Se isso for um problema para a equipe, a solução é a mesma dos blocos (guardar o valor ou a versão de referência na sobrescrita). É uma pergunta para levar à sessão, não um erro.

9. Estado derivado: foto da revista e hash · 7 min

O impresso precisa ver o texto final que mudou (D-072), e o texto final muda por vários caminhos: a matriz compartilhada, o texto-modelo, o marcador `{NOME DO PROJETO}`, um

corte que joga a tarefa para outro padrão. Em vez de rastrear cada caminho no histórico de cada tabela, o modelo guarda a **revista resolvida**: no clique «Revista Impressa Atualizada» grava uma FotoRevista (JSONB), e «o que mudou» é a diferença entre essa foto e a revista montada agora (D-077). «Validado» usa a mesma ideia com um hash: guarda o hash no clique; hash diferente hoje = não validado.

A armadilha está em produzir sempre a mesma sequência de bytes para o mesmo conteúdo. O JSONB do PostgreSQL não preserva a ordem das chaves, e um hash calculado com `json.dumps` simples depende dela. No lab:

```
revista = {"nome": "GO", "etapas": {"5EF": 1}, "ano": 2026}
foto = FotoRevista.objects.create(projeto=p, conteudo=revista); foto.refresh_from_db
()

ordem gravada : ['nome', 'etapas', 'ano']
ordem lida    : ['ano', 'nome', 'etapas']
hash ingênuo  : f79bc93b247f  09ab35bc5130     # mesmo conteúdo, hashes diferentes
hash canônico : cd6e8461a526  cd6e8461a526     # sort_keys=True, separators=(",", ":
")
```

A serialização canônica resolve a ordem das chaves, mas não a das listas: tarefas, descritores e blocos precisam sair ordenados por um campo estável (`ordem`, código) na montagem da revista. Uma lista que vem do banco sem `ORDER BY` não tem ordem garantida.

> **Pare e pense**
>
> O hash do «Validado» deve ser calculado sobre o quê: a FotoRevista guardada ou a revista montada na hora? *Responda mentalmente antes de ler o próximo parágrafo.*

Sobre a revista montada na hora, pela mesma função que monta o preview, com serialização canônica. O hash guardado no clique e o hash de agora precisam sair do mesmo código; se um vier da foto relida do JSONB e o outro da montagem, qualquer diferença de serialização vira um falso «mudou» em todos os projetos.

10. Concorrência e bloqueio de escrita · 6 min

Existem duas travas diferentes e o modelo usa as duas. A trava de linha do banco (`select_for_update()`) dura enquanto a transação roda, milissegundos, e serve para que duas requisições simultâneas não intercalem leitura e escrita. O bloqueio de edição (D-080, D-081) dura enquanto alguém está com a tela aberta, sobrevive a muitas requisições HTTP e expira em 30 s sem renovação. Esse é dado de negócio: vira tabela.

```
class Bloqueio(models.Model):
    mec = models.ForeignKey(MatrizEtapaCC, null=True, on_delete=models.CASCADE)
    projeto = models.ForeignKey(Projeto, null=True, on_delete=models.CASCADE)
    usuario = ...; expira_em = models.DateTimeField()
    class Meta:
        constraints = [
          CheckConstraint(condition=Q(mec__isnull=False) ^ Q(projeto__isnull=False),
                          name="bloqueio_um_alvo"),
          UniqueConstraint(fields=["mec"], condition=Q(mec__isnull=False),
                           name="um_bloqueio_por_mec"),
          UniqueConstraint(fields=["projeto"], condition=Q(projeto__isnull=False),
                           name="um_bloqueio_por_projeto")]

# sem alvo ou com dois alvos  -> IntegrityError "bloqueio_um_alvo"
# 2º bloqueio no mesmo mec    -> IntegrityError "um_bloqueio_por_mec"
```

Adquirir o bloqueio combina as duas travas: trava a linha do alvo, lê o bloqueio atual e decide. No lab, Bruno tentou pegar o 5EF-MT bloqueado pela Ana: recusado agora, aceito 31 s depois (expirado). A renovação automática do editor precisa rodar bem abaixo do prazo, por exemplo a cada 10 s; renovar a cada 30 s deixaria o bloqueio expirar a cada atraso de rede.

```
with transaction.atomic():
    MatrizEtapaCC.objects.select_for_update().get(pk=mec.pk)  # serializa a disputa
    b = Bloqueio.objects.filter(mec=mec).first()
    if b and b.usuario != usuario and b.expira_em > agora:
        return False
    Bloqueio.objects.update_or_create(mec=mec, defaults={
        "usuario": usuario, "expira_em": agora + timedelta(seconds=30)})
    return True
```

> **Pare e pense**
>
> O bloqueio é de escrita, não de leitura (D-081). Onde essa regra é aplicada: no banco ou na aplicação? *Responda mentalmente antes de ler o próximo parágrafo.*

Na aplicação. A tabela só registra quem tem o direito de escrever; quem impede a escrita é o servidor, que confere o bloqueio antes de aceitar cada gravação no alvo. Se a rota de salvar não conferir, a tabela não protege nada. Leitura não consulta o bloqueio, e por isso o preview continua funcionando para todos.

11. Perfis, estados e histórico · 5 min

Autenticação responde «quem é você»; autorização responde «o que você pode fazer». Os perfis agora são globais (D-073): Admin, Pedagógica, Mídias Digitais (mesmas permissões da Pedagógica) e Impresso. Não há permissão por projeto, então não há tabela Membro: cada perfil vira um `Group` do Django, e a matriz papel × ação fica no código. A documentação do Django recomenda definir um modelo de usuário próprio antes da primeira migration.

Ação	Admin	Pedagógica / Mídias	Impresso
Abrir ano, importar matriz, preencher cortes, editar textos e paleta	Sim	Sim	Não
Clicar «Validado»	Sim	Sim	Não
Clicar «Revista Impressa Atualizada» (só se validado)	Não definido	Não	Sim
Baixar planilhas, ver diff	Sim	Sim	Sim
Gerenciar usuários	Sim	Não	Não

A linha «Admin pode clicar Revista Impressa Atualizada» não foi decidida (D-083 diz que o admin tem as permissões da Pedagógica, não as do Impresso). Leve à sessão.

Ângulo de segurança: regra de estado é do servidor

«Revista Impressa Atualizada só depois do Validado» é uma regra de transição de estado. Esconder o botão no editor não é controle: qualquer pessoa com a sessão do Impresso pode chamar a rota direto. O servidor confere o perfil e o estado (hash atual igual ao `hash_validado`) antes de gravar a foto. O mesmo vale para o bloqueio e para o perfil Impresso tentar editar texto.

Se em algum lugar você escrever SQL à mão (RunSQL, relatório), valores entram como parâmetro (`%s`), nunca por f-string. O ORM parametriza; a injeção de SQL volta quando você concatena texto.

O histórico por tabela (django-simple-history, D-082) deixou de alimentar o aviso ao impresso, que vem da foto. Ele serve à auditoria: quem mudou o quê e quando. Cada `save()` grava uma cópia da linha numa tabela espelho com `history_date`, `history_user` e `history_type` (+, ~, −), e `diff_against` devolve os campos alterados. Atenção: `bulk_create` e `update()` não chamam `save()`, então a clonagem do ano não gera histórico por padrão; a biblioteca oferece `bulk_create_with_history` para isso (confira na documentação da versão instalada).

12. Checklist para cada tabela e relacionamento · 2 min

Seis perguntas para cada tabela

1. Qual é a identidade e o que mais é UNIQUE? O ano faz parte do escopo?
2. Quais colunas aceitam NULL, e o que o NULL significa em cada uma?
3. O que acontece com os filhos quando esta linha é apagada? Ela deveria ser arquivada em vez de apagada?
4. Algum valor aqui pode ser calculado a partir de outras tabelas? Se pode, por que guardar?
5. Ela entra no clone do ano? Em que ordem?
6. Quem escreve, quem lê, e precisa de bloqueio ou de histórico?

Quatro perguntas para cada relacionamento

1. Quantos de cada lado, no mínimo e no máximo?

2. A ligação tem atributos? Então é entidade.

3. As duas pontas precisam ser do mesmo ano? Então FK composta ou validação.

4. A regra depende de olhar várias linhas (um ativo por ano, um bloqueio por alvo)? Então UNIQUE parcial, transação ou trigger.

13. O que NÃO estudar agora · 1 min

Tema	Por que fica para depois
CTE recursiva, closure table	Sem herança entre projetos, não há corrente de ancestrais para percorrer
Migração de dados do Mongo	2026 começa do zero (D-076)
Índices, EXPLAIN, particionamento	Volume pequeno (dezenas de parceiros por ano). PK, UNIQUE e FK já criam ou pedem índice
Triggers e procedures	Só se uma regra entre tabelas não couber em FK composta nem em validação
select_related / prefetch_related	Entra ao escrever as rotas, não ao modelar

Leitura de apoio, depois das questões: restrições do Django (`docs.djangoproject.com/en/stable/ref/models/constraints/`), campos do PostgreSQL no Django (`docs.djangoproject.com/en/stable/ref/contrib/postgres/fields/`), tipos JSON do PostgreSQL (`postgresql.org/docs/current/datatype-json.html`, trecho sobre ordem das chaves) e restrições de FK (`postgresql.org/docs/current/ddl-constraints.html`).

Prática no papel · 15 min

Use `docs/Modelo_ER_Revista.pdf` impresso e a planilha `Cortes dos Padrões por Etapa e Parceiro.xlsx`.

Roteiro

1. Na página 1 do ER, escreva ao lado de cada seta o `on_delete` que você escolheria (CASCADE, PROTECT, SET_NULL).

2. Circule cada par de FKs que precisa apontar para o mesmo ano (há pelo menos quatro).

3. Numere as tabelas na ordem em que o clone do ano as copia. Risque as que não entram no clone.

4. Na aba 3EM-LP da planilha, calcule os grupos à mão pela regra da seção 4 e confira com a coluna Grupo.

5. Para cada tabela, responda a pergunta 4 do checklist (algum valor é calculável?).

Questões

Responda por escrito e mande no chat. Sem gabarito aqui; a correção é na conversa. Em ordem crescente de dificuldade.

1. Na aba 3EM-LP os cortes são: MT (200, 275, 325); MS (224, 324, 374); PE (225, 270, 305); CE, GO e PI (225, 275, 325); ES (250, 300, 350); BA, MA, PB, RN, RO e Porto Alegre (250, 300, 375); SE (275, 325, 375). Numere os grupos. Depois, se PE mudar para (225, 275, 325), quais números de grupo mudam para os outros parceiros? O que isso diz sobre guardar o grupo numa coluna?

2. Um colega propõe Corte(parceiro, ano, etapa, cc, valores, escala_max, grupo, qtd_padroes). Diga, coluna por coluna, o que sai da tabela e por qual motivo (forma normal violada ou dado derivado).

3. SALVADOR, 5EF-LP, cortes (175, 210, 228). Uma tarefa ocupa o intervalo 200 a 225. Em qual padrão ela aparece na revista de SALVADOR, pela regra D-058? Em qual padrão a mesma tarefa aparece para GO (125, 175, 225)? Onde essa informação fica no banco?

4. A restrição de Etapa foi escrita como `UNIQUE(codigo)` e ninguém percebeu em 2026. Descreva o que acontece no primeiro clique em «abrir 2027» e o que o usuário vê, considerando que o clone roda em `transaction.atomic()`.

5. GO perde todos os cortes em 2026 (projeto arquivado) e duas semanas depois volta a ter cortes. Quantas linhas de Projeto GO 2026 existem no fim, com que valores de `arquivado` e de `nome`? Qual restrição permite isso e qual impediria um erro de cascata que criasse dois ativos?

6. Alguém configura o Projeto ES 2027 apontando para a matriz «ES» de 2026. Com FKs simples, o que acontece na revista? Qual mecanismo da seção 7 recusa isso, e por que um `CheckConstraint` não conseguiria?

7. Um dev implementa o «Validado» assim: no clique, `hash_validado = sha256(json.dumps(foto.conteudo))` lendo a última FotoRevista; na tela, compara com `sha256(json.dumps(montar_revista(p)))`. Logo depois do clique, todos os projetos aparecem como não validados. Explique as duas causas possíveis e a correção.

8. Ana edita o 5EF-MT da matriz comunitária; o editor renova o bloqueio a cada 10 s; a última renovação foi às 9:59:55 e a aba travou. Bruno quer editar o 9EF-LP da mesma matriz às 10:00:05; Carla quer o 5EF-MT às 10:00:10 e de novo às 10:00:30. Quem consegue o quê, e por quê? O que mudaria se o bloqueio fosse da matriz inteira?

9. Linha do tempo do projeto PE: dia 1 Validado; dia 2 o impresso clica «Revista Impressa Atualizada» (foto F1); dia 3 corrigem a matriz comunitária; dia 4 Validado de novo pelo botão de lote; dia 5 o nome do projeto muda; dia 6 Validado. No dia 7, o que o impresso vê como mudança e o botão dele está habilitado? E se o dia 6 não tivesse acontecido?

10. O intervalo mínimo do 5EF-MT na matriz muda de 125 para 150. O projeto MS tinha sobrescrito com 100; o projeto PE não sobrescreveu. Qual o efetivo de cada um? O impresso fica sabendo da mudança de PE? E da de MS? Que coluna faltaria para avisar a pedagógica sobre MS, como já acontece com os blocos?

11. Alguém roda `DELETE FROM matriz WHERE nome = 'CE'` no psql, com o Projeto CE 2026 apontando para ela. O `on_delete=PROTECT` do modelo impede? O que impede de fato, e o que aconteceria se a FK tivesse sido declarada com `on_delete=models.DB_CASCADE`?

12. Desafio. Em 2027 o clone copia matrizes e cortes, e os projetos renascem pela cascata (D-085). No rascunho atual, a escolha de matriz e de texto-modelo mora no Projeto (D-084). (a) Que matriz o Projeto CE 2027 recebe ao nascer, se nada mais for feito? (b) Isso fere qual decisão do levantamento? (c) Proponha uma mudança no modelo que preserve a escolha de matriz entre anos sem clonar projetos, diga em que tabela ela mora, o que entra no clone e qual restrição de mesmo ano ela exige.